

# CSD\_02 — System Software and Compilers

**Paper-I, Unit 5** — *entirely degree-only*. Official syllabus wording:

*"Assemblers, macros, loaders, linkers and editors. Lexical analysis, Syntax analysis — different parsing techniques, semantic analysis, Error detection, Optimization and code generation."*

The shared-core files do not touch this unit. Nothing in the CS (Diploma) or Computer Forensic syllabi overlaps it. Every hour spent here pays into exactly one paper — but it pays heavily.

## Why this matters / where it appears

Together with algorithm analysis ([CSD\\_01](#)), this is one of the **two highest-yield blocks** in the whole Degree elective. The evidence from the 2025 papers:

1. **Paper-I Section C (15 marks) offered "Phases of a compiler, lexical analysis to code generation; task and output of each."** That is a *bankable* 15-marker — a fixed, learnable answer with a diagram, and it recurs year after year in every compiler syllabus on earth.
2. **Paper-I Section A Q10 (5 marks) asked "Role of a linker in the compilation process."** A cheap, predictable 5-marker.
3. **The MCQ paper devotes Q31–50 to "digital logic, computer architecture and system software"** — 20% of a 200-mark paper. 2025 MCQ items included *"Which loader function resolves external references?"* and *"Which parsing technique uses a stack and a parsing table?"*

**The study-method warning (same as algorithms):** this is *worked-mechanism* territory, not *read-about* territory. FIRST/FOLLOW sets, an LL(1) table, an LR(0) item-set collection, a two-pass assembler symbol table, three-address-code translation — these are **mechanical skills** that must be performed by hand, repeatedly, spread over days. Nobody builds an SLR table for the first time in the exam hall. Work every table in this file from a blank page at least twice.

## Tick-box checklist

Tick a box only when you can reproduce the worked example *from a blank page*.

| §  | Topic                                                                              | Depth              | Done                     |
|----|------------------------------------------------------------------------------------|--------------------|--------------------------|
| 1  | System vs application software; the software hierarchy                             | 5-mark             | <input type="checkbox"/> |
| 2  | Assemblers: one-pass vs two-pass; MOT/POT/ST/LT/base table; <b>worked two-pass</b> | 10/15-mark         | <input type="checkbox"/> |
| 3  | Macros and macro processors: definition, expansion, nested, conditional            | 5/10-mark          | <input type="checkbox"/> |
| 4  | Loaders: compile-and-go, absolute, relocating, direct-linking, dynamic, bootstrap  | MCQ + 5/10         | <input type="checkbox"/> |
| 5  | Linkers: static vs dynamic, relocation, resolution, linker vs loader               | 5-mark, asked 2025 | <input type="checkbox"/> |
| 6  | Editors: line, stream, screen, structure                                           | 5-mark             | <input type="checkbox"/> |
| 7  | <b>Phases of a compiler</b> — six phases + tables, traced on one statement         | 15-mark, bankable  | <input type="checkbox"/> |
| 8  | Lexical analysis: tokens/lexemes/patterns, regex, NFA→DFA, lex                     | 10-mark            | <input type="checkbox"/> |
| 9  | Syntax: CFG, ambiguity, left recursion, left factoring                             | 10-mark            | <input type="checkbox"/> |
| 10 | <b>Top-down parsing: LL(1), worked FIRST/FOLLOW + parse table</b>                  | 15-mark            | <input type="checkbox"/> |
| 11 | <b>Bottom-up parsing: shift-reduce, LR(0)/SLR/CLR/LALR, worked item sets</b>       | MCQ + 15-mark      | <input type="checkbox"/> |

|    |                                                                                      |                |                          |
|----|--------------------------------------------------------------------------------------|----------------|--------------------------|
| 12 | Semantic analysis: SDD, attribute grammars, type checking                            | 10-mark        | <input type="checkbox"/> |
| 13 | Intermediate code: three-address, quadruples/triples/indirect triples                | 10-mark        | <input type="checkbox"/> |
| 14 | Optimisation: basic blocks, flow graph, DAG, loop opt, peephole, register allocation | <b>15-mark</b> | <input type="checkbox"/> |
| 15 | Code generation: target code, instruction selection, register allocation             | 10-mark        | <input type="checkbox"/> |
| 16 | Error detection and recovery per phase                                               | 5/10-mark      | <input type="checkbox"/> |

# 1. System software vs application software

## Concept — in plain English first

**Application software** is software you run *to get a job done that you care about* — a word processor, a browser, a game, a payroll program. **System software** is software that exists *to make other software possible* — it manages the hardware and provides the platform and the tools that applications are written in, translated by, and run on. The user buys the machine for the applications; the system software is the invisible scaffolding underneath.

An analogy: the application is the play the audience came to see; the system software is the theatre, the stage crew, the lighting rig and the translator in the wings.

## Key points

|            | System software                                                                 | Application software                           |
|------------|---------------------------------------------------------------------------------|------------------------------------------------|
| Purpose    | Manages/controls hardware; platform for other software                          | Solves a specific user problem                 |
| Runs       | Automatically / in the background                                               | When the user launches it                      |
| Written by | Systems programmers (close to the machine)                                      | Application developers                         |
| Depends on | The hardware directly                                                           | The system software beneath it                 |
| Examples   | OS, assembler, compiler, interpreter, linker, loader, device drivers, utilities | MS Word, Chrome, VLC, a payroll system, a game |
| Generality | General-purpose (serves all applications)                                       | Special-purpose (one problem domain)           |

**The software hierarchy (draw this as a stack of layers):**

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                |                                                                                                                                               |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> +-----+             APPLICATION SOFTWARE             +-----+             SYSTEM SOFTWARE                     - Language processors: assembler,              compiler, interpreter, preprocessor        - Linkers and loaders                        - Operating system + device drivers          - Utilities (editors, debuggers,               antivirus)                             +-----+             HARDWARE                       +-----+ </pre> | <pre> Word, browser, games, payroll translate programs build &amp; load executables manage resources supporting tools CPU, memory, I/O </pre> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|

**Language processors** are the system-software family this unit is about:

| Processor      | Input → Output                                                        | Note                                                                              |
|----------------|-----------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| Assembler      | Assembly language → machine code (object)                             | One-to-one (mostly) mnemonic → opcode                                             |
| Compiler       | High-level source → machine/object code, <b>whole program at once</b> | Produces a standalone executable; errors reported before running                  |
| Interpreter    | High-level source → executes <b>statement by statement</b>            | No object file; slower; errors found at the failing line                          |
| Preprocessor   | Source with directives → expanded source                              | Macro expansion, file inclusion (C <code>#include</code> , <code>#define</code> ) |
| Cross-compiler | Source on machine A → code for machine B                              | For embedded/mobile targets                                                       |

**Compiler vs interpreter — a guaranteed MCQ and a common 5-marker.** Compiler: translates the entire program once, produces an object/executable file, faster execution, harder to debug, errors all reported at compile time (C, C++). Interpreter: translates and executes line by line, no object file, slower execution, easier interactive debugging, error reported at the offending line (early Python, BASIC). Java uses **both**: `javac` compiles to bytecode, the JVM interprets/JIT-compiles it.

## Likely exam questions

1. Differentiate between system software and application software with examples. **[5]**
2. Differentiate between a compiler and an interpreter. **[5]**
3. What is a language processor? Describe assembler, compiler, interpreter and preprocessor. **[10]**

## MCQ traps

- An **assembler** translates *assembly* language, not a high-level language (that is a compiler).
- An **interpreter** produces **no** object file; a compiler does.
- The OS is system software; a device driver is system software; **antivirus is a utility (system software)**, not an application in this taxonomy.
- Java is compiled to bytecode **then** interpreted — "compiled *and* interpreted".

## 2. Assemblers

### Concept — in plain English first

An **assembler** turns assembly language (human-readable mnemonics like `ADD`, `MOV`, `LOAD`) into machine code (binary opcodes the CPU executes). Mostly it is a simple one-to-one translation: `ADD` → the binary opcode for add. The one thing that makes it non-trivial is the **forward reference** problem.

Consider:

```
JMP  LOOP          ; where is LOOP? we don't know yet — it appears later
...
LOOP:  LOAD X
```

When the assembler reads the `JMP LOOP` instruction, it has not yet seen the label `LOOP`, so it does not know its address. It cannot fill in the jump target. This is a **forward reference**, and how an assembler copes with it is the whole distinction between one-pass and two-pass designs.

## Key points — the databases (tables) an assembler keeps

These are the tables you must be able to name and describe — they come up as a 5-marker on their own:

| Table             | Full name            | Contents                                                                                                                                 | Fixed or built?            |
|-------------------|----------------------|------------------------------------------------------------------------------------------------------------------------------------------|----------------------------|
| <b>MOT</b>        | Machine Opcode Table | Mnemonic → binary opcode, instruction length, format                                                                                     | <b>Fixed</b> (predefined)  |
| <b>POT</b>        | Pseudo-Opcode Table  | Assembler directives ( <b>START</b> , <b>END</b> , <b>DS</b> , <b>DC</b> , <b>EQU</b> , <b>USING</b> ) and the routine that handles each | <b>Fixed</b>               |
| <b>ST</b>         | Symbol Table         | Each label → its address (value), length, type                                                                                           | <b>Built</b> during Pass 1 |
| <b>LT</b>         | Literal Table        | Each literal (e.g. <b>=5</b> ) → the address assigned to it                                                                              | <b>Built</b> during Pass 1 |
| <b>Base Table</b> | Base Register Table  | Which registers are base registers and their contents (from <b>USING/DROP</b> )                                                          | Built                      |
| <b>LC</b>         | Location Counter     | Not a table — a running counter of the current address                                                                                   | Runs in both passes        |

### Common assembler directives (pseudo-ops):

| Directive             | Meaning                                                      |
|-----------------------|--------------------------------------------------------------|
| <b>START</b> <i>n</i> | Program begins; set LC = <i>n</i>                            |
| <b>END</b>            | End of source; may name the entry point                      |
| <b>DS</b> <i>n</i>    | Define Storage — reserve <i>n</i> units, no initial value    |
| <b>DC</b> <i>v</i>    | Define Constant — reserve and initialise with value <i>v</i> |
| <b>EQU</b>            | Equate a symbol to a value (assembly-time constant)          |
| <b>USING/DROP</b>     | Declare/undeclare a base register                            |
| <b>LTORG</b>          | Assemble the literal pool here                               |

## 2.1 One-pass vs two-pass assemblers

|                    | One-pass                                                                                                  | Two-pass                                                                             |
|--------------------|-----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Passes over source | 1                                                                                                         | 2                                                                                    |
| Forward references | Handled by <b>back-patching</b> (keep a table of unfilled locations, fix them once the symbol is defined) | Resolved naturally: Pass 1 builds the full symbol table before Pass 2 generates code |
| Speed              | Faster (one read)                                                                                         | Slower (two reads)                                                                   |
| Memory             | Needs a table of pending forward references                                                               | Needs the whole symbol table                                                         |
| Simplicity         | Trickier logic (patching)                                                                                 | Cleaner, easier to understand and extend                                             |
| Used when          | Load-and-go, memory tight, speed critical                                                                 | The standard general-purpose design                                                  |

**One-pass idea:** when you meet **JMP** **LOOP** and **LOOP** is undefined, emit the instruction with a blank

address and add "location of this blank" to a forward-reference list attached to `LOOP`. When `LOOP` is later defined, walk the list and back-patch every blank with the now-known address.

## 2.2 The two-pass assembler — what each pass does

### Pass 1 — "define the symbols".

- Initialise the Location Counter (LC) from `START`.
- Read each line. For each, look up its length in the MOT/POT and **increment the LC**.
- When a line has a **label**, enter (label → current LC) into the **Symbol Table**.
- Collect **literals** into the **Literal Table** and assign them addresses (at `LTORG` or `END`).
- Process directives that affect the LC (`DS`, `DC`, `EQU`).
- **Output**: a complete Symbol Table + Literal Table, and an intermediate copy of the source with LC values attached. **No machine code is generated yet.**

### Pass 2 — "generate the code".

- Re-read the (intermediate) source with the LC known for every line.
- For each instruction: look up the opcode in the MOT, look up every operand symbol in the Symbol Table (now fully populated, so forward references just work), assemble the binary instruction.
- Handle `DC/DS` by emitting/reserving data; place the literal pool.
- **Output**: the object program (machine code + relocation & linking information) and the assembly listing.

### Worked two-pass example — trace this by hand

Assume word-addressed memory, each instruction 1 word. Source program:

```

      START 100
      LOAD  A
      ADD   B
      SUB   ONE
      STORE A
      JMP   DONE      ; forward reference — DONE not yet seen
A      DC   5
B      DC   3
ONE     DC   1
DONE    STORE B
      END
```

**PASS 1 — assign addresses, build the Symbol Table.** LC starts at 100 (from `START 100`).

| LC  | Label | Statement | Action in Pass 1                     |
|-----|-------|-----------|--------------------------------------|
| 100 |       | LOAD A    | instruction, len 1 → LC becomes 101  |
| 101 |       | ADD B     | LC → 102                             |
| 102 |       | SUB ONE   | LC → 103                             |
| 103 |       | STORE A   | LC → 104                             |
| 104 |       | JMP DONE  | forward ref noted; LC → 105          |
| 105 | A     | DC 5      | enter <b>A = 105</b> in ST; LC → 106 |
| 106 | B     | DC 3      | enter <b>B = 106</b> ; LC → 107      |
| 107 | ONE   | DC 1      | enter <b>ONE = 107</b> ; LC → 108    |
| 108 | DONE  | STORE B   | enter <b>DONE = 108</b> ; LC → 109   |

### Symbol Table after Pass 1:

| Symbol | Address | Type                |
|--------|---------|---------------------|
| A      | 105     | data                |
| B      | 106     | data                |
| ONE    | 107     | data                |
| DONE   | 108     | instruction (label) |

**PASS 2 — generate object code.** Suppose the MOT gives opcodes: LOAD = 01, ADD = 02, SUB = 03, STORE = 04, JMP = 05. Every operand address now comes straight from the completed Symbol Table — the forward reference to **DONE** (address 108) is no longer a problem.

| LC  | Statement | Opcode | Operand addr | Object code                   |
|-----|-----------|--------|--------------|-------------------------------|
| 100 | LOAD A    | 01     | 105          | 01 105                        |
| 101 | ADD B     | 02     | 106          | 02 106                        |
| 102 | SUB ONE   | 03     | 107          | 03 107                        |
| 103 | STORE A   | 04     | 105          | 04 105                        |
| 104 | JMP DONE  | 05     | 108          | 05 108 ← forward ref resolved |
| 105 | DC 5      | —      | —            | 00005 (constant)              |
| 106 | DC 3      | —      | —            | 00003                         |
| 107 | DC 1      | —      | —            | 00001                         |
| 108 | STORE B   | 04     | 106          | 04 106                        |

**This is the answer an examiner wants for a 10/15-mark "explain the working of a two-pass assembler with an example" question:** the two-column pass 1 (address assignment + symbol table) followed by the pass 2 code generation table, with the forward reference explicitly pointed out.

## Likely exam questions

1. What is an assembler? Explain the tables (MOT, POT, ST, LT) it maintains. **[5]**
2. Differentiate between a one-pass and a two-pass assembler. **[5]**
3. Explain the working of a two-pass assembler with a suitable example, showing the symbol table. **[10/15]**
4. What is a forward reference? How does a two-pass assembler handle it, and how does a one-pass assembler handle it (back-patching)? **[10]**

## MCQ traps

- The **Symbol Table** and **Literal Table** are *built* by the assembler; the **MOT** and **POT** are *predefined/fixed*.
- **Pass 1 assigns addresses and builds the symbol table; Pass 2 generates the object code.** (Reversed in a distractor.)
- Forward references are the *reason* two passes exist.
- The **Location Counter (LC)** runs in **both** passes.

- Literals (=5) go in the **Literal Table**; symbols (labels) go in the **Symbol Table**.

## 3. Macros and macro processors

### Concept — in plain English first

A **macro** is a named shorthand for a block of assembly (or source) text. You **define** it once, then **call** it by name wherever you want that block; the **macro processor** replaces each call with a fresh copy of the block — this replacement is called **expansion**. It is *textual substitution done before assembly*, so there is no call/return overhead — the code is physically copied in (**inline**), unlike a subroutine which is called at run time.

**Macro vs subroutine — a favourite comparison:**

|            | Macro                                      | Subroutine                          |
|------------|--------------------------------------------|-------------------------------------|
| Mechanism  | Code copied inline at <b>assembly time</b> | One copy, <b>called at run time</b> |
| Overhead   | No call/return cost                        | Call, parameter push, return cost   |
| Code size  | <b>Larger</b> (copy per call)              | Smaller (single copy)               |
| Speed      | Faster (no call overhead)                  | Slower (call overhead)              |
| Handled by | Macro processor / assembler                | CPU at run time (CALL/RET)          |

### Key points — how a macro processor works

Four data structures / functions:

| Component                           | Role                                                          |
|-------------------------------------|---------------------------------------------------------------|
| <b>MDT</b> (Macro Definition Table) | Stores the body of each defined macro                         |
| <b>MNT</b> (Macro Name Table)       | Maps macro name → its entry (index) in the MDT                |
| <b>ALA</b> (Argument List Array)    | Maps formal parameters → actual arguments during an expansion |
| <b>MDTP / MDI</b>                   | Macro Definition Table Pointer / index used while expanding   |

Two logical stages: **(1) macro definition processing** — recognise **MACRO ... MEND**, store the body in the MDT, record the name in the MNT; **(2) macro call expansion** — on a call, look up the name in the MNT, fetch the body from the MDT, substitute the actual arguments (via the ALA) for the formals, and emit the resulting text into the source stream that the assembler then sees.

### Worked example — definition and expansion

**Definition:**

```
MACRO
INCR  &X, &Y          ; &X, &Y are formal (dummy) parameters
LOAD  &X
ADD   &Y
STORE &X
MEND
```

**Call:** INCR A, B

**Expansion** (the macro processor substitutes  $&X \rightarrow A$ ,  $&Y \rightarrow B$  and copies the body inline):

```
LOAD  A
ADD   B
STORE A
```

A second call `INCR P, Q` expands to a *second, independent copy* with P and Q. That copying is why macros grow code size.

## 3.1 Nested macros

A macro whose body **calls another macro** (or itself). The processor expands the outer call, and each inner macro call encountered during expansion is itself expanded.

```
MACRO
ADD2  &A, &B
LOAD  &A
ADD   &B
MEND

MACRO
SUM3  &X, &Y, &Z
ADD2  &X, &Y      ; nested call
ADD   &Z
MEND
```

`SUM3 P, Q, R` first expands to `ADD2 P, Q + ADD R`, and `ADD2 P, Q` further expands to `LOAD P + ADD Q`. Final: `LOAD P / ADD Q / ADD R`.

- **Nested definition:** a macro defined *inside* another (the inner one is only defined once the outer is first called).
- **Nested call / recursive macro:** a macro that calls another or itself — needs a stack in the processor and a termination condition (via conditional expansion) to avoid infinite expansion.

## 3.2 Conditional macro expansion

The body can decide *what* to expand based on the arguments, using **macro-time conditional directives** — `AIF` (assemble if), `AGO` (unconditional branch), `SET` symbols (assembly-time variables), and sequence symbols (labels like `.LOOP`). This is what lets one macro generate different code for different calls, and lets recursive macros terminate.

```
MACRO
GEN    &N
LCLA   &CTR                      ; local assembly-time counter
&CTR   SETA  1
.LOOP  AIF   (&CTR GT &N).DONE   ; if CTR > N, jump to .DONE
WORD   &CTR
&CTR   SETA  &CTR+1
AGO    .LOOP
.DONE  MEND
```

`GEN 3` generates `WORD 1 / WORD 2 / WORD 3`. The `AIF` is evaluated **at assembly time**, not run time — no conditional appears in the final code.

**Conditional assembly vs run-time if:** conditional macro expansion (`AIF`) chooses *which code to generate*; a run-time `if` chooses *which generated code to execute*. A guaranteed distinction MCQ.

## Likely exam questions

1. What is a macro? Differentiate a macro from a subroutine. [5]
2. Explain macro definition and expansion with an example. [5/10]
3. Explain the data structures (MNT, MDT, ALA) used by a macro processor. [10]
4. What are nested and conditional macros? Give examples of each. [10]

## MCQ traps

- A macro is expanded at **assembly (compile) time**, inline; a subroutine is invoked at **run time**.
- Macros **increase** code size but **reduce** call overhead.
- **AIF/AGO** are evaluated at **assembly time** (conditional assembly), not run time.
- **MNT** = names, **MDT** = definitions (bodies), **ALA** = argument substitution.

## 4. Loaders

### Concept — in plain English first

A **loader** is the system program that takes an object program (produced by the assembler/compiler, possibly linked) and **puts it into main memory, prepares it to run, and hands control to it**. Its four classical functions are the four things it might have to do:

| Function   | What it does                                                              |
|------------|---------------------------------------------------------------------------|
| Allocation | Reserve memory space for the program                                      |
| Linking    | Resolve references between separately compiled modules (external symbols) |
| Relocation | Adjust address-dependent locations for the actual load address            |
| Loading    | Physically place the code and data into memory and start it               |

Different **loader schemes** perform different subsets of these four, which is exactly how the five types are told apart.

### Key points — the loader types (learn this table cold; MCQ gold)

| Loader type                             | Allocation | Linking | Relocation | Loading       | Idea                                                                                                                                                                                    |
|-----------------------------------------|------------|---------|------------|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Compile-and-go</b> (assemble-and-go) | —          | —       | —          | ✓             | Assembler places code directly in memory and runs it; no object file. Simple, but re-assembles every run and wastes the assembler's memory.                                             |
| <b>Absolute loader</b>                  | prog.      | prog.   | —          | ✓<br>(loader) | Assembler does everything; loader just places code at the <b>fixed</b> address the assembler assumed. Simple & fast, but the programmer must fix addresses and manage linking manually. |
| <b>Relocating (BSS) loader</b>          | loader     | —       | ✓          | ✓             | Assembler emits <b>relocation bits/records</b> : loader can place the program <b>anywhere</b> and fix relocatable addresses. Allows independent module assembly.                        |

|                                          |          |          |          |          |                                                                                                                                                                                         |
|------------------------------------------|----------|----------|----------|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Direct-linking loader</b>             | loader   | ✓        | ✓        | ✓        | The <b>general-purpose</b> loader: does all four functions, using ESD (external symbols), TXT (code), RLD (relocation) and END records. Allows multiple independently compiled modules. |
| <b>Dynamic loading / dynamic linking</b> | run time | run time | run time | run time | Modules/routines are loaded/linked <b>only when first called</b> , at run time. Saves memory (unused routines never loaded); DLLs / shared objects.                                     |

Plus the special one:

- **Bootstrap loader:** the tiny program (in ROM/firmware) that runs at power-on and loads the *first* program — the operating-system loader — into memory. It "pulls itself up by its own bootstraps": a small loader loads a bigger loader that loads the OS. This is the origin of the word "**boot**".

**Direct-linking loader — the object record types (worth naming):**

| Record                                      | Meaning                                                                                        |
|---------------------------------------------|------------------------------------------------------------------------------------------------|
| <b>ESD</b> — External Symbol Dictionary     | Symbols this module <b>defines</b> (for others) and symbols it <b>references</b> (from others) |
| <b>TXT</b> — Text                           | The actual machine code and data                                                               |
| <b>RLD</b> — Relocation & Linkage Directory | Which locations need adjusting for relocation/linking                                          |
| <b>END</b>                                  | End of the module; may give the entry point                                                    |

## Advantages / disadvantages worth a line

| Scheme         | Advantage                                                       | Disadvantage                                                         |
|----------------|-----------------------------------------------------------------|----------------------------------------------------------------------|
| Compile-and-go | Simplest                                                        | Re-assemble each run; assembler occupies memory; no separate modules |
| Absolute       | Small, fast loader                                              | Programmer fixes addresses; manual linking; not relocatable          |
| Relocating     | Program can load anywhere; independent modules                  | More complex loader; relocation overhead                             |
| Direct-linking | Full generality; libraries; independent compilation             | Largest, most complex loader                                         |
| Dynamic        | Saves memory; one shared copy of a library; smaller executables | Run-time overhead on first call; "DLL hell" version issues           |

## Likely exam questions

1. What is a loader? Explain its four functions (allocation, linking, relocation, loading). **[5]**
2. Explain the different types of loaders. **[10]** (*the syllabus names them explicitly*)
3. Differentiate absolute and relocating loaders. **[5]**
4. What is a bootstrap loader? Explain the booting sequence. **[5]**

5. Explain dynamic loading and dynamic linking; state their advantages. [10]

## MCQ traps

- "Which loader function resolves external references between object files?" → **Linking**. (This was a 2025 MCQ.)
- **Relocation** = adjust addresses for the load location; **linking** = resolve external symbol references. Do not confuse them.
- An **absolute loader** does **no** relocation; a **relocating loader** does.
- The **bootstrap loader** loads the OS; it is the reason it is called "booting".
- Dynamic linking happens at **run time**, not build time.

## 5. Linkers

### Concept — in plain English first

Large programs are split into many source files, each compiled separately into an **object module**. Each module may *use* names (functions, variables) that are *defined* in another module — an **external reference**. The **linker** is the program that takes all these object modules (plus library modules) and stitches them into one executable, doing two jobs: **symbol resolution** (match every external reference to exactly one definition) and **relocation** (each module was compiled as if it started at address 0; the linker lays them out one after another and fixes every address accordingly).

This is exactly the 2025 Q10 answer (5 marks): *"The linker runs after the compiler/assembler and before the loader. It takes one or more object files, resolves external references — a call in one module to a function defined in another — and relocates the modules into a single address space, producing an executable."*

### Key points

The build pipeline (memorise the order):

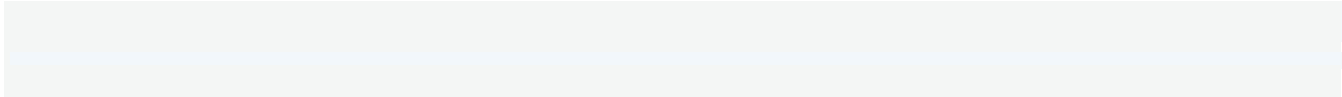
```
source → preprocessor → compiler → assembler → object file(s) → LINKER → executable →  
LOADER → running process
```

Linker vs loader — the distinction examiners test:

|        | Linker                                                             | Loader                                           |
|--------|--------------------------------------------------------------------|--------------------------------------------------|
| When   | <b>Build time</b> (before execution)                               | <b>Load time</b> (start of execution)            |
| Input  | Object modules + libraries                                         | The executable                                   |
| Job    | Resolve external references, relocate, combine into one executable | Bring the image into memory, set it up, start it |
| Output | An executable file                                                 | A running process                                |

Two linking strategies:

|                             | Static linking                                   | Dynamic linking                                                 |
|-----------------------------|--------------------------------------------------|-----------------------------------------------------------------|
| When library code is joined | At <b>link time</b> — copied into the executable | At <b>run time</b> — loaded on demand (DLL / <code>.so</code> ) |



**The document editing process — four operations:** *traveling* (moving to the point of edit), *editing* (insert/delete/replace), *viewing* (mapping the internal document onto the screen), and *display* (updating the screen). A 5-marker rarely goes deeper than the type table above.

## Likely exam questions

- 1. What is a text editor? Explain its different types. [5]
- 2. Differentiate a line editor from a screen editor. [5]

## MCQ traps

- `sed` is a **stream** editor (non-interactive); `vi` is a **screen** editor.
- A **structure editor** is grammar-aware (syntax-directed).

# 7. Phases of a compiler — the bankable 15-marker

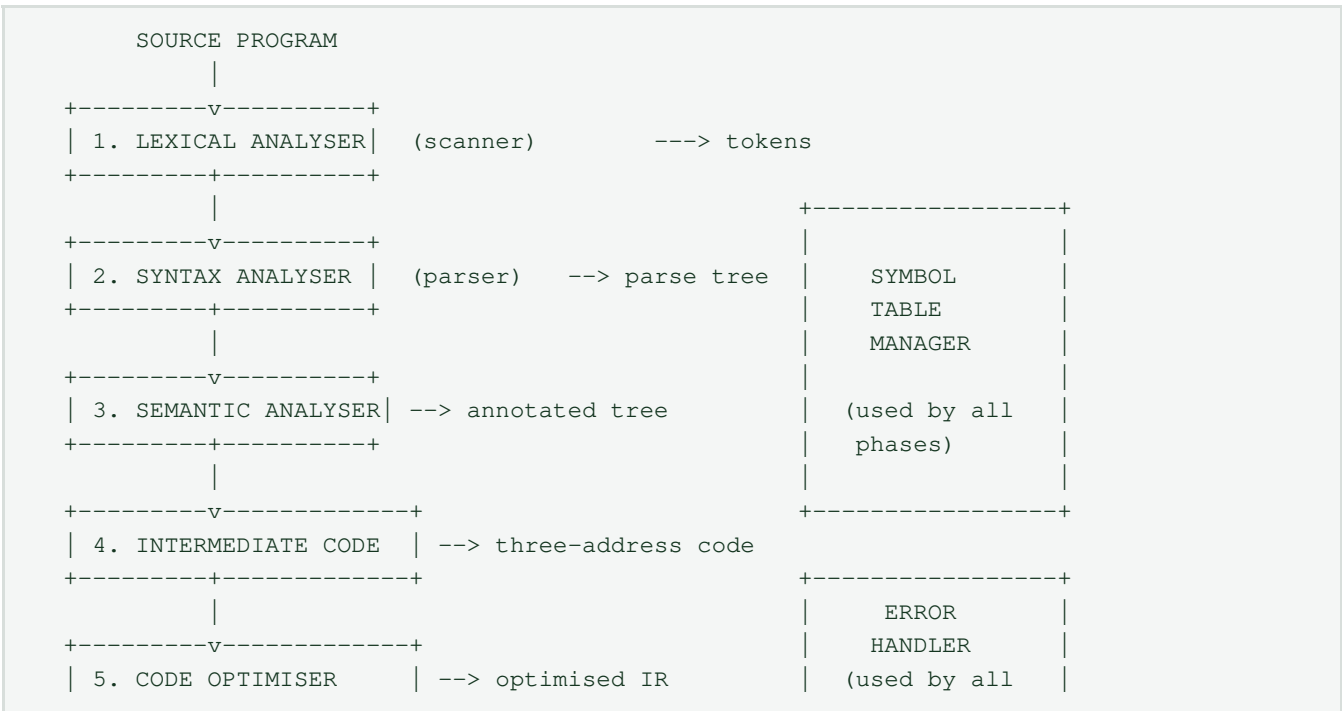
## Concept — in plain English first

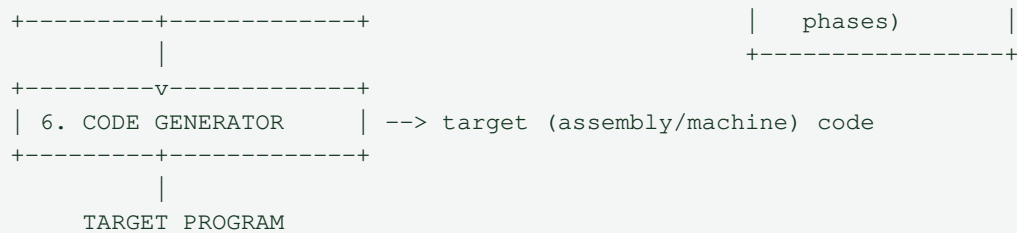
A compiler turns high-level source into target code. It does not do this in one leap; it does it as a **pipeline of phases**, each taking the output of the previous one and refining it, the way an assembly line turns raw material into a finished product. Two groupings:

- **Analysis (front end)** — *understand* the source: lexical, syntax, semantic analysis. Produces the intermediate representation. Machine-independent.
- **Synthesis (back end)** — *build* the target: intermediate code, optimisation, code generation. Depends on the target machine.

Across all phases run two shared modules: the **symbol table manager** and the **error handler**.

## The six phases — the diagram to reproduce





(Draw the six boxes vertically in a column; draw the symbol table and error handler as two boxes on the side connected to all phases. That side-connection is worth a mark on its own — say "both are used by every phase".)

## One statement traced through every phase

Trace the single statement: `position = initial + rate * 60`

| Phase                       | What it does                                                                                                     | Output for this statement                                                                                                                                                                                         |
|-----------------------------|------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>1. Lexical analysis</b>  | Group characters into <b>tokens</b> (lexeme, token-class); enter identifiers in symbol table; discard whitespace | <code>id1 = id2 + id3 * 60</code> where <code>id1=position</code> , <code>id2=initial</code> , <code>id3=rate</code> , and <code>=</code> , <code>+</code> , <code>*</code> , <code>60</code> (number) are tokens |
| <b>2. Syntax analysis</b>   | Check tokens against the grammar; build a <b>parse/syntax tree</b>                                               | Tree: <code>=</code> at root, <code>id1</code> left, <code>+</code> right; under <code>+</code> : <code>id2</code> and <code>*</code> ; under <code>*</code> : <code>id3</code> and <code>60</code>               |
| <b>3. Semantic analysis</b> | Type-check; insert coercions. <code>60</code> is int, <code>rate</code> is real → insert <code>inttoreal</code>  | Tree with <code>inttoreal(60)</code> inserted before the <code>*</code>                                                                                                                                           |
| <b>4. Intermediate code</b> | Emit three-address code                                                                                          | <code>t1 = inttoreal(60) → t2 = id3 * t1 → t3 = id2 + t2 → id1 = t3</code>                                                                                                                                        |
| <b>5. Code optimisation</b> | Simplify. <code>inttoreal(60)</code> is compile-time constant → fold; drop the redundant temporary               | <code>t1 = id3 * 60.0 → id1 = id2 + t1</code>                                                                                                                                                                     |
| <b>6. Code generation</b>   | Emit target assembly, allocate registers                                                                         | <code>LDF R2, id3 / MULF R2, R2, #60.0 / LDF R1, id2 / ADDF R1, R1, R2 / STF id1, R1</code>                                                                                                                       |

**This trace is the heart of the 15-mark answer.** Learn it cold: state each phase, its *task*, and its *output* on this one statement. Add the two side modules (symbol table, error handler) and a one-line front-end/back-end grouping, and it is a full-marks answer.

## Front end vs back end / passes

- **Front end** (lexical + syntax + semantic + intermediate code gen): depends on the *source language*, independent of the target machine.
- **Back end** (optimisation + code generation): depends on the *target machine*, independent of the source language. This split is why one front end + many back ends builds a portable compiler (and one back end + many front ends lets many languages target one machine — the LLVM idea).
- **Pass vs phase**: a *phase* is a logical stage; a *pass* is one complete read of the program. A multi-phase compiler may run several phases in a single pass.

## Likely exam questions

1. Explain the phases of a compiler, from lexical analysis to code generation, describing the task and output of each. **[15]** (exactly the 2025 Q18 — trace one statement through all six)

2. Differentiate the analysis phase and the synthesis phase (front end vs back end). [5]
3. What is the role of the symbol table and the error handler across the compiler phases? [5]
4. Differentiate between a phase and a pass. [5]

## MCQ traps

- Order: **lexical** → **syntax** → **semantic** → **intermediate code** → **optimisation** → **code generation**.
- **Type checking** is the job of **semantic analysis**, not syntax analysis.
- The **parse tree** is produced by **syntax analysis**.
- The symbol table and error handler are used by **all** phases, not one.
- Optimisation and code generation are the **back end** (machine-dependent).

## 8. Lexical analysis (the scanner)

### Concept — in plain English first

The lexical analyser reads the raw source characters and groups them into meaningful chunks called **tokens** — like reading a sentence and recognising words rather than letters. It also throws away whitespace and comments, and enters identifiers/constants into the symbol table. It is the only phase that reads the source character by character, so keeping it separate makes the rest of the compiler simpler and faster.

**Three terms you must not confuse:**

| Term    | Meaning                                                           | Example                               |
|---------|-------------------------------------------------------------------|---------------------------------------|
| Token   | A category / class name                                           | identifier, keyword, number, relop    |
| Lexeme  | The actual character string matched                               | count, while, 3.14, <=                |
| Pattern | The rule describing all lexemes of a token (a regular expression) | letter(letter\ digit)* for identifier |

So in `count <= 10`, the lexemes are `count`, `<=`, `10`; the tokens are `id`, `relop`, `number`; the patterns are the regexes that recognise each class.

### Key points — regular expressions and finite automata

Token patterns are described by **regular expressions**, recognised by **finite automata**:

| Regex operator    | Meaning                         |
|-------------------|---------------------------------|
| <code>a\ b</code> | a or b (alternation)            |
| <code>ab</code>   | a followed by b (concatenation) |
| <code>a*</code>   | zero or more a (Kleene star)    |
| <code>a+</code>   | one or more a                   |
| <code>a?</code>   | optional a                      |

Examples: identifier = `letter (letter | digit)*`; unsigned integer = `digit+`; real number = `digit+ . digit+`.

## The scanner-generator pipeline (lex/flex):

```
regular expressions → NFA (Thompson's construction) → DFA (subset construction) →  
minimised DFA → scanner code
```

- **NFA** (Nondeterministic Finite Automaton): may have  $\epsilon$ -moves and multiple transitions on one symbol; easy to build from a regex.
- **DFA** (Deterministic Finite Automaton): exactly one transition per symbol, no  $\epsilon$ -moves; easy to execute (a simple table lookup). A real scanner runs a DFA.
- **NFA → DFA** is the **subset construction**: each DFA state is a *set* of NFA states reachable together. This determinisation is the standard exam procedure.

## Worked NFA → DFA (subset construction)

Regex `(a|b)*abb` — the classic. NFA states 0..? ; here is the compact result of subset construction (each DFA state = set of NFA states, computed via  $\epsilon$ -closure and move):

| DFA state         | = NFA set        | on <b>a</b> | on <b>b</b> |
|-------------------|------------------|-------------|-------------|
| <b>A</b> (start)  | {0,1,2,4,7}      | B           | C           |
| <b>B</b>          | {1,2,3,4,6,7,8}  | B           | D           |
| <b>C</b>          | {1,2,4,5,6,7}    | B           | C           |
| <b>D</b>          | {1,2,4,5,6,7,9}  | B           | <b>E</b>    |
| <b>E</b> (accept) | {1,2,4,5,6,7,10} | B           | C           |

The procedure to state (and this earns marks): **(1)** compute  $\epsilon$ -closure of the start state → DFA start state; **(2)** for each DFA state and each input symbol, compute **move** then  **$\epsilon$ -closure** → the next DFA state; **(3)** repeat until no new states; **(4)** a DFA state is accepting if its set contains an NFA accepting state.

**lex/flex** is the tool: you write `pattern { action }` rules; lex builds the DFA and generates C code for the scanner. `yylval`, longest-match and rule-order (first rule wins on a tie) are the practical rules.

## Likely exam questions

1. What is lexical analysis? Differentiate between token, lexeme and pattern with examples. [5]
2. Explain the role of the lexical analyser. How are regular expressions and finite automata used? [10]
3. Convert the regular expression `(a|b)*abb` to an NFA and then to a DFA using subset construction. [10/15]
4. What is lex? Describe the structure of a lex program. [5]

## MCQ traps

- **Token** = class, **lexeme** = actual string, **pattern** = the regex. (Constantly swapped.)
- A real scanner runs a **DFA**, not an NFA.
- **Subset construction** converts NFA→DFA; **Thompson's construction** converts regex→NFA.
- Lexical analysis removes **whitespace and comments** and enters symbols in the symbol table.
- Regular languages **cannot** count matching parentheses — that is why syntax analysis (CFG), not lexical analysis, handles nesting.

## 9. Syntax analysis: grammars

### Concept — in plain English first

The parser checks that the stream of tokens forms a *grammatically valid* program and builds a **parse tree** showing its structure — the way you diagram a sentence to show subject, verb and object. Grammar is described by a **context-free grammar (CFG)**, because regular expressions cannot express nesting (balanced parentheses, nested blocks).

### Key points — CFG

A CFG is a 4-tuple  $G = (V, T, P, S)$ :  $V$  = non-terminals (variables),  $T$  = terminals (tokens),  $P$  = productions (rules),  $S$  = start symbol.

Example expression grammar:

```
E → E + T | T
T → T * F | F
F → ( E ) | id
```

- **Derivation:** replacing non-terminals by production right-hand sides from  $S$  to a string of terminals. **Leftmost derivation** always expands the leftmost non-terminal; **rightmost** the rightmost.
- **Parse tree:** the tree of a derivation; leaves are terminals, internal nodes non-terminals.

### 9.1 Ambiguity

A grammar is **ambiguous** if some string has **two or more distinct parse trees** (equivalently, two leftmost derivations). Example:  $E \rightarrow E + E \mid E * E \mid id$  parses `id + id * id` two ways — one that multiplies first, one that adds first. Ambiguity is bad because it makes the *meaning* undefined.

**Fix:** rewrite the grammar to encode precedence and associativity (as the  $E/T/F$  grammar above does —  $*$  binds tighter because it is deeper in the grammar), or use disambiguating rules.

**Ambiguity is undecidable in general** — there is no algorithm to test an arbitrary CFG for ambiguity. A likely MCQ.

### 9.2 Left recursion removal

A grammar is **left-recursive** if  $A \Rightarrow A \alpha$  (a non-terminal can derive itself as its own leftmost symbol), e.g.  $E \rightarrow E + T$ . **Top-down (recursive-descent/LL) parsers loop forever** on left recursion, so it must be removed.

**Rule.** Replace  $A \rightarrow A \alpha \mid \beta$  with  
 $A \rightarrow \beta A' \quad A' \rightarrow \alpha A' \mid \epsilon$

**Worked:**  $E \rightarrow E + T \mid T$  becomes

```
E → T E'
E' → + T E' | ε
```

and  $T \rightarrow T * F \mid F$  becomes  $T \rightarrow F T', T' \rightarrow * F T' \mid \epsilon$ .

## 9.3 Left factoring

When two productions for the same non-terminal share a common prefix, a predictive parser cannot decide which to use by looking at one token. **Left factoring** pulls out the common prefix.

**Rule.** Replace  $A \rightarrow \alpha \beta_1 \mid \alpha \beta_2$  with  
 $A \rightarrow \alpha A' \quad A' \rightarrow \beta_1 \mid \beta_2$

**Worked (the dangling-else / if grammar):**

```
S → i E t S | i E t S e S | a
```

share the prefix  $i E t S$ , so factor:

```
S → i E t S S' | a  
S' → e S | ε
```

## Likely exam questions

1. Define a context-free grammar. What is a derivation and a parse tree? [5]
2. What is an ambiguous grammar? Show that  $E \rightarrow E + E \mid E * E \mid id$  is ambiguous. [10]
3. What is left recursion? Remove it from  $E \rightarrow E + T \mid T, T \rightarrow T * F \mid F$ . [10]
4. What is left factoring? Left-factor  $S \rightarrow iEtS \mid iEtSeS \mid a$ . [5]

## MCQ traps

- Ambiguity = **two parse trees** for one string (not "two productions").
- **Left recursion** breaks **top-down** parsers, not bottom-up ones (LR parsers handle it fine).
- Left factoring is needed for **predictive/LL** parsing.
- Testing an arbitrary CFG for ambiguity is **undecidable**.

## 10. Top-down parsing — LL(1) with worked FIRST/FOLLOW and parse table

### Concept — in plain English first

**Top-down** parsing builds the parse tree from the **root down**, trying to derive the input from the start symbol, always expanding the **leftmost** non-terminal. A **predictive parser** does this without backtracking by peeking at the next input token and using a table to decide which production to apply.

**LL(1)** = scan **Left**-to-right, produce a **Leftmost** derivation, using **1** token of lookahead.

To build the table you need two sets for the grammar: **FIRST** (which terminals can begin a string derived from a symbol) and **FOLLOW** (which terminals can appear immediately after a non-terminal).

### The rules

**FIRST(X):**

1. If  $X$  is a terminal,  $\text{FIRST}(X) = \{X\}$ .
2. If  $X \rightarrow \epsilon$ , add  $\epsilon$  to  $\text{FIRST}(X)$ .

3. If  $X \rightarrow Y_1 Y_2 \dots Y_k$ : add  $\text{FIRST}(Y_1)\{\epsilon\}$ ; if  $Y_1$  can be  $\epsilon$ , also add  $\text{FIRST}(Y_2)$ , and so on; if *all*  $Y_i$  can derive  $\epsilon$ , add  $\epsilon$ .

**FOLLOW(A):**

- 1. Put **\$** (end marker) in FOLLOW(start symbol).
- 2. If  $A \rightarrow \alpha B \beta$ : add  $\text{FIRST}(\beta)\{\epsilon\}$  to FOLLOW(B).
- 3. If  $A \rightarrow \alpha B$ , or  $A \rightarrow \alpha B \beta$  where  $\beta \Rightarrow \epsilon$ : add FOLLOW(A) to FOLLOW(B).

**Worked example — the classic expression grammar**

Grammar (already left-recursion-removed):

```
E  → T E'
E' → + T E' | ε
T  → F T'
T' → * F T' | ε
F  → ( E ) | id
```

**FIRST sets:**

| Symbol | FIRST                |
|--------|----------------------|
| F      | { (, id }            |
| T      | { (, id } (FIRST(F)) |
| E      | { (, id } (FIRST(T)) |
| T'     | { *, ε }             |
| E'     | { +, ε }             |

**FOLLOW sets:**

| Non-terminal | FOLLOW          | Why                                                                                     |
|--------------|-----------------|-----------------------------------------------------------------------------------------|
| E            | { }, \$ }       | \$ (start) and ) from $F \rightarrow ( E )$                                             |
| E'           | { }, \$ }       | FOLLOW(E) ( $E \rightarrow T E'$ )                                                      |
| T            | { +, ), \$ }    | $\text{FIRST}(E')\{\epsilon\} = \{+\}$ , plus FOLLOW(E) since $E' \Rightarrow \epsilon$ |
| T'           | { +, ), \$ }    | FOLLOW(T) ( $T \rightarrow F T'$ )                                                      |
| F            | { *, +, ), \$ } | $\text{FIRST}(T')\{\epsilon\} = \{*\}$ , plus FOLLOW(T) since $T' \Rightarrow \epsilon$ |

**LL(1) parse table.** For each production  $A \rightarrow \alpha$ : put  $A \rightarrow \alpha$  in cell  $[A, a]$  for every  $a$  in  $\text{FIRST}(\alpha)$ ; if  $\epsilon \in \text{FIRST}(\alpha)$ , also put it in  $[A, b]$  for every  $b$  in FOLLOW(A).

|    | id                   | +                         | *                       | (                    | )                         | \$                        |
|----|----------------------|---------------------------|-------------------------|----------------------|---------------------------|---------------------------|
| E  | $E \rightarrow T E'$ |                           |                         | $E \rightarrow T E'$ |                           |                           |
| E' |                      | $E' \rightarrow + T E'$   |                         |                      | $E' \rightarrow \epsilon$ | $E' \rightarrow \epsilon$ |
| T  | $T \rightarrow F T'$ |                           |                         | $T \rightarrow F T'$ |                           |                           |
| T' |                      | $T' \rightarrow \epsilon$ | $T' \rightarrow * F T'$ |                      | $T' \rightarrow \epsilon$ | $T' \rightarrow \epsilon$ |

|   |      |  |  |         |  |  |
|---|------|--|--|---------|--|--|
| F | F→id |  |  | F→( E ) |  |  |
|---|------|--|--|---------|--|--|

The grammar is LL(1) because **no cell has two entries** (no conflict).

### Worked parse of `id + id * id` (stack-driven predictive parsing)

Stack starts \$ E, input `id + id * id $`. At each step, if the stack top is a terminal, match it; if a non-terminal, replace it using the table cell [top, current-input].

| Stack        | Input           | Action                                  |
|--------------|-----------------|-----------------------------------------|
| \$ E         | id + id * id \$ | $E \rightarrow T E'$                    |
| \$ E' T      | id + id * id \$ | $T \rightarrow F T'$                    |
| \$ E' T' F   | id + id * id \$ | $F \rightarrow id$                      |
| \$ E' T' id  | id + id * id \$ | match id                                |
| \$ E' T'     | + id * id \$    | $T' \rightarrow \epsilon$               |
| \$ E'        | + id * id \$    | $E' \rightarrow + T E'$                 |
| \$ E' T +    | + id * id \$    | match +                                 |
| \$ E' T      | id * id \$      | $T \rightarrow F T'$                    |
| \$ E' T' F   | id * id \$      | $F \rightarrow id \rightarrow$ match id |
| \$ E' T'     | * id \$         | $T' \rightarrow * F T'$                 |
| \$ E' T' F * | * id \$         | match *                                 |
| \$ E' T' F   | id \$           | $F \rightarrow id \rightarrow$ match id |
| \$ E' T'     | \$              | $T' \rightarrow \epsilon$               |
| \$ E'        | \$              | $E' \rightarrow \epsilon$               |
| \$           | \$              | accept                                  |

This full worked example — FIRST, FOLLOW, table, and a parse — is a complete 15-mark answer.

**LL(1) condition (state it):** a grammar is LL(1) iff for every non-terminal A with productions  $A \rightarrow \alpha \mid \beta$ : (1)  $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$ , and (2) if  $\beta \Rightarrow \epsilon$  then  $\text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \emptyset$ . Left recursion and un-factored common prefixes both violate this — which is why you remove them first.

## Likely exam questions

1. Compute FIRST and FOLLOW for the grammar  $E \rightarrow TE'$ ,  $E' \rightarrow +TE' \mid \epsilon$ ,  $T \rightarrow FT'$ ,  $T' \rightarrow FT' \mid \epsilon$ ,  $F \rightarrow (E) \mid id$ . [10\*]
2. Construct the LL(1) parsing table for the above grammar and parse `id + id * id`. [15]
3. What is a predictive parser? State the LL(1) condition. [5]
4. What is recursive-descent parsing? How does it relate to LL(1)? [5]

## MCQ traps

- **LL(1)** = Left-to-right scan, Leftmost derivation, 1 lookahead.

- A grammar with **left recursion** is **not** LL(1) (and neither is one with common prefixes).
- The presence of **any multiply-defined table entry** means the grammar is **not** LL(1).
- FIRST is about what can *begin*; FOLLOW about what can *come after*.  $\$$  only ever appears in FOLLOW.

## 11. Bottom-up parsing — shift-reduce and LR family

### Concept — in plain English first

**Bottom-up** parsing builds the tree from the **leaves up** to the root — it reads tokens and *reduces* substrings back to non-terminals until it reaches the start symbol. It effectively runs a **rightmost derivation in reverse**. The dominant technique is **shift-reduce** parsing, driven by a stack and, in the LR family, a parsing table. LR parsers are strictly more powerful than LL parsers — they handle left recursion and a larger class of grammars.

### 11.1 Shift-reduce parsing

Four actions on a stack + input:

| Action | Meaning                                                                                   |
|--------|-------------------------------------------------------------------------------------------|
| Shift  | Push the next input token onto the stack                                                  |
| Reduce | The top of the stack matches a production RHS (a <b>handle</b> ); pop it and push the LHS |
| Accept | Stack = start symbol, input exhausted → success                                           |
| Error  | No valid move                                                                             |

A **handle** is a substring that matches the right side of a production **and** whose reduction is a step in the reverse rightmost derivation. Finding handles is the whole problem.

**Two conflicts to name:**

- **Shift-reduce conflict:** the parser cannot decide whether to shift or reduce (classic: the dangling-else).
- **Reduce-reduce conflict:** two different productions could be reduced.

**Worked shift-reduce parse of  $\text{id} * \text{id}$  with  $E \rightarrow E+T \mid T, T \rightarrow T * F \mid F, F \rightarrow \text{id}$**

| Stack     | Input      | Action                           |
|-----------|------------|----------------------------------|
| \$        | id * id \$ | shift                            |
| \$ id     | * id \$    | reduce $F \rightarrow \text{id}$ |
| \$ F      | * id \$    | reduce $T \rightarrow F$         |
| \$ T      | * id \$    | shift                            |
| \$ T *    | id \$      | shift                            |
| \$ T * id | \$         | reduce $F \rightarrow \text{id}$ |
| \$ T * F  | \$         | reduce $T \rightarrow T * F$     |
| \$ T      | \$         | reduce $E \rightarrow T$         |

**SLR(1) table** (reduce entries placed under FOLLOW; here FOLLOW(S)={\$}, FOLLOW(C)={c,d,\$}):

| State | c       | d       | \$      | S | C |
|-------|---------|---------|---------|---|---|
| 0     | s3      | s4      |         | 1 | 2 |
| 1     |         |         | acc     |   |   |
| 2     | s3      | s4      |         |   | 5 |
| 3     | s3      | s4      |         |   | 6 |
| 4     | r(C→d)  | r(C→d)  | r(C→d)  |   |   |
| 5     |         |         | r(S→CC) |   |   |
| 6     | r(C→cC) | r(C→cC) | r(C→cC) |   |   |

$sN$  = shift and go to state N;  $rX$  = reduce by production X; **acc** = accept; blank = error. **No cell has a conflict**, so the grammar is SLR(1). Reproducing the item sets I0–I6, the goto arrows, and this table is a full 15-mark answer.

**SLR vs LALR vs CLR difference in one line:** SLR uses the whole FOLLOW set to decide reductions (crude, can conflict); CLR carries an exact lookahead in every item (precise but many states); LALR merges CLR states with identical cores, keeping the precise-enough lookahead in far fewer states.

## Likely exam questions

1. What is shift-reduce parsing? Explain shift, reduce, accept, error with a worked parse of `id*id`. [10]
2. What is a handle? What are shift-reduce and reduce-reduce conflicts? [5]
3. Construct the LR(0) item sets and the SLR parsing table for `S→CC, C→cC|d`. [15]
4. Compare LR(0), SLR, LALR and CLR parsers on power and table size. [10]
5. Differentiate top-down and bottom-up parsing (LL vs LR). [5]

## MCQ traps

- "Which parsing technique uses a stack and a parsing table?" → LR (shift-reduce) parsing (also LL(1)); the 2025 MCQ answer was the stack-and-table parser.
- **LR** = rightmost derivation **in reverse**; **LL** = leftmost derivation.
- Power: **CLR(1) is the most powerful, LR(0) the least**; **LALR** is used by yacc.
- LR parsers **handle left recursion**; LL parsers do not.
- The dangling-else is the classic **shift-reduce conflict**.

# 12. Semantic analysis

## Concept — in plain English first

Syntax analysis checks the *shape* of the program; semantic analysis checks its *meaning* — the things a grammar cannot express: is `x` declared before use? Do the types match? Is a function called with the

right number of arguments? Are array indices used on arrays? This is the last front-end phase, and its main tool is the **syntax-directed definition** attached to the grammar.

## Key points

---

- **Syntax-Directed Definition (SDD):** a CFG with **attributes** attached to grammar symbols and **semantic rules** attached to productions. It says *what* to compute, not the order.
- **Attributes:**
- **Synthesized** — value computed from the node's **children** (flows *up* the tree). Example: the type/value of an expression.
- **Inherited** — value computed from the node's **parent/siblings** (flows *down/across*). Example: passing a declared type to a list of identifiers.
- **S-attributed grammar:** uses **only synthesized** attributes; evaluable bottom-up in a single post-order pass (fits LR parsing).
- **L-attributed grammar:** each inherited attribute depends only on things to its **left**; evaluable in one left-to-right pass (fits top-down/LL parsing). Every S-attributed grammar is L-attributed.
- **Type checking:** verify operand types are compatible; insert **type coercions/conversions** (e.g. `int→real`); report type errors. Uses a **type system** and the **symbol table** (which holds each identifier's type, scope, storage).
- **Type expressions and equivalence:** structural vs name equivalence of types (a common MCQ).

**Worked idea.** For `int x; x = x + 3.5;` the semantic analyser looks up `x` (type `int`) in the symbol table, sees `3.5` is real, and either flags a type error or inserts a coercion — this is a semantic check the grammar alone could never make.

## Likely exam questions

---

1. What is semantic analysis? What kinds of errors does it catch that syntax analysis cannot? [5]
2. Differentiate synthesized and inherited attributes with an example. [10]
3. What is a syntax-directed definition? Differentiate S-attributed and L-attributed grammars. [10]
4. Explain type checking and type coercion in a compiler. [5]

## MCQ traps

---

- **Type checking is semantic analysis**, not syntax analysis.
- **Synthesized = from children (up); inherited = from parent (down).**
- **S-attributed → LR/bottom-up; L-attributed → LL/top-down.**
- "Undeclared variable" and "type mismatch" are **semantic** errors caught after parsing.

# 13. Intermediate code generation

## Concept — in plain English first

---

Between the front end and the back end sits an **intermediate representation (IR)** — a machine-neutral "assembly for an abstract machine". It decouples the source language from the target: one IR lets many front ends share many back ends. The most common IR is **three-address code**.

## Key points — three-address code (TAC)

Each instruction has **at most one operator and at most three addresses** (two operands + one result):

$x = y \text{ op } z$ . Complex expressions are broken up using **temporaries** (t1, t2, ...).

**Example.**  $a = b * c + b * d$  becomes:

```
t1 = b * c
t2 = b * d
t3 = t1 + t2
a  = t3
```

**TAC statement types:** assignments ( $x = y \text{ op } z$ ,  $x = \text{op } y$ ,  $x = y$ ), copy, unconditional jump (`goto L`), conditional jump (`if x relop y goto L`), procedure call (`param x, call p,n`), indexed ( $x = a[i]$ ,  $a[i] = x$ ), pointer ( $x = *p$ ,  $*p = x$ ).

## The three representations of TAC

| Representation          | How it stores each instruction                                                   | Property                                                                                          |
|-------------------------|----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| <b>Quadruples</b>       | (op, arg1, arg2, result) — result named explicitly                               | Easy to move/reorder code (optimisation-friendly); uses temporaries                               |
| <b>Triples</b>          | (op, arg1, arg2) — result referred to by the <b>triple's own position/number</b> | More compact (no temporaries), but <b>moving code renumbers references</b> — hard to optimise     |
| <b>Indirect triples</b> | A list of pointers to triples, plus the triple table                             | Reorder by shuffling the pointer list, not the triples → combines compactness with reorderability |

## Worked — represent $a = b * c + b * d$ three ways

TAC:

```
(0) t1 = b * c
(1) t2 = b * d
(2) t3 = t1 + t2
(3) a  = t3
```

**Quadruples:**

| # | op | arg1 | arg2 | result |
|---|----|------|------|--------|
| 0 | *  | b    | c    | t1     |
| 1 | *  | b    | d    | t2     |
| 2 | +  | t1   | t2   | t3     |
| 3 | =  | t3   |      | a      |

**Triples** (results are the triple numbers in parentheses):

| #   | op | arg1 | arg2 |
|-----|----|------|------|
| (0) | *  | b    | c    |

|     |   |     |     |
|-----|---|-----|-----|
| (1) | * | b   | d   |
| (2) | + | (0) | (1) |
| (3) | = | a   | (2) |

**Indirect triples:** a pointer list [35, 36, 37, 38] indexing a triple table containing the four triples above; to reorder, permute the pointer list only.

**The trade-off to state:** quadruples waste space on explicit temporaries but are easy to optimise/reorder; triples are compact but position-dependent (reordering breaks references); indirect triples restore reorderability by adding a pointer indirection. Guaranteed comparison MCQ.

**Other IRs to name:** postfix (Polish) notation; syntax trees / DAGs (see §14).

## Likely exam questions

1. What is intermediate code? Why is it used (portability, front/back-end decoupling)? [5]
2. Write three-address code for  $a = b * c + b * d$  (or  $x = (a+b) * (c+d)$ ). [5]
3. Explain quadruples, triples and indirect triples; represent a given expression in all three. [10]
4. Differentiate triples and indirect triples. [5]

## MCQ traps

- Three-address code has **at most three addresses** and **one operator** per instruction.
- **Triples** refer to results by **position**; **quadruples** name an explicit **result** field.
- **Indirect triples** ease code reordering during optimisation (triples do not).
- Intermediate code exists mainly for **portability** (one IR, many targets).

# 14. Code optimisation

## Concept — in plain English first

Optimisation transforms the code so it runs faster and/or uses less space, **without changing what it computes**. It is the machine-independent + machine-dependent polishing between IR and final code. Key correctness rule: an optimisation must be **safe** (preserve program meaning) before it can be profitable.

## Key points — classifications

- **Local optimisation:** within a single basic block.
- **Global optimisation:** across basic blocks, using the flow graph and data-flow analysis.
- **Machine-independent** (on the IR) vs **machine-dependent** (register allocation, peephole on target code).

## 14.1 Basic blocks and the flow graph

- **Basic block:** a maximal straight-line sequence of instructions with **one entry (the leader)** and **one exit** — no jumps in except at the top, no jumps out except at the bottom.
- **Leaders:** (1) the first instruction; (2) any target of a jump; (3) any instruction immediately after a jump. Each leader begins a new block.
- **Flow graph:** nodes = basic blocks, edges = possible control flow between them. Data-flow analysis runs over this graph to enable global optimisation. **Loops** show up as cycles / back edges in the flow graph.

## 14.2 The standard machine-independent optimisations

| Optimisation                           | What it does                                                           | Example (before → after)                                                 |
|----------------------------------------|------------------------------------------------------------------------|--------------------------------------------------------------------------|
| Constant folding                       | Evaluate constant expressions at compile time                          | <code>x = 3 * 4 → x = 12</code>                                          |
| Constant propagation                   | Replace a variable known to be constant by that constant               | <code>a=5; b=a+2 → b=7</code>                                            |
| Common subexpression elimination (CSE) | Compute a repeated expression once                                     | <code>t1=b*c; ...; t2=b*c → reuse t1</code>                              |
| Dead-code elimination                  | Remove code whose result is never used                                 | remove <code>x = y+1</code> if <code>x</code> is never read              |
| Copy propagation                       | After <code>x=y</code> , use <code>y</code> in place of <code>x</code> | enables more dead-code removal                                           |
| Strength reduction                     | Replace an expensive op with a cheaper one                             | <code>x*2 → x+x</code> or <code>x&lt;&lt;1; i*4</code> in a loop → add 4 |
| Algebraic simplification               | Use identities                                                         | <code>x+0 → x, x*1 → x, x*0 → 0</code>                                   |
| Loop-invariant code motion             | Move computations that don't change in the loop out of it              | hoist <code>y = a*b</code> out of a loop over <code>i</code>             |
| Loop unrolling                         | Replicate the body to cut loop-control overhead                        |                                                                          |
| Induction-variable elimination         | Simplify variables that change linearly with the loop counter          |                                                                          |

## 14.3 The DAG (Directed Acyclic Graph)

Within a basic block, build a **DAG** of the expressions: leaves are variables/constants, interior nodes are operators, and **identical subexpressions share a node**. This exposes common subexpressions (they become the *same node*) and dead code (nodes no live variable points to). A DAG for `a = b*c + b*c` has a single `b*c` node used twice — CSE falls straight out of it.

## 14.4 Peephole optimisation

A **machine-dependent** local optimisation that slides a small "window" (a few instructions) over the target code and replaces recognised inefficient patterns:

| Pattern removed              | Example                                                             |
|------------------------------|---------------------------------------------------------------------|
| Redundant load/store         | <code>STORE R, a</code> then <code>LOAD a, R</code> → drop the load |
| Unreachable code             | code after an unconditional jump with no label                      |
| Jumps to jumps               | <code>goto L1; ... L1: goto L2</code> → <code>goto L2</code>        |
| Algebraic/strength on target | <code>MUL R, 2</code> → <code>SHL R, 1</code>                       |
| Redundant instructions       | <code>ADD R, 0; MUL R, 1</code> removed                             |

## 14.5 Register allocation

Registers are the fastest storage but few; the optimiser decides which values live in registers to minimise memory traffic. Classic method: build a **register-interference graph** (nodes = values, edges = values live at the same time) and **graph-colour** it with k colours (k = number of registers); values that cannot get a colour are **spilled** to memory. Register allocation is a machine-dependent back-end optimisation.

## Likely exam questions

1. What is code optimisation? Differentiate local and global (machine-independent vs -dependent). [5]
2. What is a basic block? How do you find leaders and construct a flow graph? [10]
3. Explain constant folding, CSE, dead-code elimination, strength reduction and loop-invariant code motion with examples. [15]
4. What is a DAG? How does it help detect common subexpressions? [10]
5. What is peephole optimisation? List the transformations it performs. [5]

## MCQ traps

- A **basic block** has **one entry and one exit**.
- **Constant folding** = compute at compile time; **constant propagation** = substitute a known constant.
- **Strength reduction**: `x*2` → `x+x` / shift.
- **Peephole** optimisation is **machine-dependent** and **local** (small window).
- A **DAG** exposes common subexpressions (shared nodes) — it is acyclic.
- Loop-invariant code motion **hoists** invariant computations *out* of the loop.

## 15. Code generation

### Concept — in plain English first

The back end's final phase turns the (optimised) IR into actual target code — machine or assembly language. It must pick real instructions, assign registers, and choose addressing modes, while keeping the code correct and, ideally, short and fast.

## Key points — the issues a code generator must handle

| Issue                            | Meaning                                                                                                                       |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Input                            | The optimised IR + symbol table                                                                                               |
| Target program form              | Absolute machine code, relocatable object, or assembly                                                                        |
| Instruction selection            | Map each IR operation to the best target instruction(s) — one IR op may map to several instructions, or several IR ops to one |
| Register allocation & assignment | Which values in registers (fast, scarce) vs memory; which specific register                                                   |
| Evaluation order                 | Order of computation affects how many registers are needed                                                                    |
| Addressing modes                 | Use the target's modes efficiently                                                                                            |

A **simple code-generation algorithm** uses a **register descriptor** (what each register currently holds) and an **address descriptor** (where each variable's current value lives) to decide, for each TAC instruction, which register to use (*getreg*) and whether a load/store is needed.

**Worked.** TAC  $t = a + b$  with a simple machine might generate:

```
MOV a, R0
ADD b, R0
MOV R0, t
```

and the code generator would try to keep  $t$  in R0 so the next instruction using  $t$  avoids reloading.

**Cost of an instruction** = 1 (base) + added cost of each operand's addressing mode; the generator minimises total cost. Optimal code generation for general expressions is NP-hard, but for expression trees the **Sethi–Ullman labelling algorithm** produces code using the fewest registers (worth naming).

## Likely exam questions

1. What is code generation? What are the main issues in designing a code generator? [10]
2. Explain instruction selection and register allocation. [5]
3. Generate target code for the three-address code of a given expression. [10]
4. What are register and address descriptors? [5]

## MCQ traps

- Code generation is the **back end**, **machine-dependent**.
- **Register allocation** (which values in registers) vs **register assignment** (which specific register) — sometimes distinguished.
- Optimal code generation is **NP-hard** in general; **Sethi–Ullman** is optimal for expression trees.

## 16. Error detection and recovery (per phase)

## Concept

A compiler must not stop at the first error — it should report as many genuine errors as possible in one run, at the right phase, without cascading false errors. Each phase detects its own kind of error.

| Phase                           | Errors it detects                     | Examples                                                                                       |
|---------------------------------|---------------------------------------|------------------------------------------------------------------------------------------------|
| Lexical                         | Malformed tokens                      | Illegal character, unterminated string/comment, <code>12abc</code>                             |
| Syntax                          | Violations of grammar                 | Missing <code>;</code> , unbalanced <code>)</code> , <code>if (x</code> with no <code>)</code> |
| Semantic                        | Meaning violations the grammar allows | Undeclared variable, type mismatch, wrong number of arguments, array/scalar misuse             |
| Intermediate/Code gen           | (rare) target constraints             | Value too large for a register/field                                                           |
| Run time (not a compiler phase) | Division by zero, null dereference    | Detected while running                                                                         |

## Error-recovery strategies in the parser

| Strategy          | Idea                                                                                                                                                      |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Panic mode        | Discard input tokens until a <b>synchronising token</b> ( <code>;</code> , <code>)</code> is found, then resume. Simple, never loops, but may skip a lot. |
| Phrase-level      | Locally repair the input (insert a missing <code>;</code> , delete an extra token) and continue                                                           |
| Error productions | Augment the grammar with rules for common errors so the parser recognises and reports them                                                                |
| Global correction | Find the smallest set of changes to make the program legal (theoretically ideal; too costly in practice)                                                  |

**A good compiler's error report:** states the phase, the location (line/column), and a clear message; recovers so that subsequent code is still checked; and avoids reporting spurious cascade errors.

## Likely exam questions

1. What types of errors are detected at each phase of a compiler? [5]
2. Explain the error-recovery strategies used in a parser (panic mode, phrase level, error productions, global correction). [10]
3. Differentiate compile-time errors and run-time errors. [5]

## MCQ traps

- "Undeclared variable" / "type mismatch" → **semantic**; "missing semicolon" → **syntax**; "illegal character" → **lexical**.
- **Panic-mode** recovery skips to a **synchronising token**.
- Division by zero is a **run-time** error, not detected by the compiler.

## Exam armour — one-page recall for this unit

| Question type                            | Instant answer                                                       |
|------------------------------------------|----------------------------------------------------------------------|
| Passes in an assembler                   | Pass 1: addresses + symbol table; Pass 2: object code                |
| Loader that resolves external references | Linking function / direct-linking loader                             |
| Loads the OS at power-on                 | Bootstrap loader                                                     |
| Linker vs loader                         | Linker: build-time, resolve+relocate; Loader: load-time, place+start |
| Six compiler phases                      | Lexical → Syntax → Semantic → Intermediate → Optimisation → Code gen |
| Token / lexeme / pattern                 | class / actual string / regex                                        |
| NFA → DFA                                | Subset construction                                                  |
| Removes left recursion for               | Top-down (LL) parsing                                                |
| LL(1)                                    | Left scan, Leftmost deriv, 1 lookahead; needs FIRST/FOLLOW           |
| Most powerful LR                         | CLR/LR(1); yacc uses LALR                                            |
| Type checking phase                      | Semantic analysis                                                    |
| Synthesized vs inherited                 | children→up / parent→down                                            |
| TAC result by position                   | Triples                                                              |
| Basic block                              | one entry, one exit                                                  |
| Peephole                                 | machine-dependent, small window                                      |
| Panic-mode recovery                      | skip to synchronising token                                          |

**Closing note.** Compilers is the single most reliable 15-marker on Paper-I and 20% of the MCQ system-software block. The compiler-phases trace (§7) and one worked parse — either the LL(1) table (§10) or the SLR item sets (§11) — should be automatic from a blank page. Get those two things cold and this unit is banked.